

```

• double speed;

• public void accelerate(){
• //code to accelerate
• }

• public void turnLeft(){
• //code to turn left
• }
• public void turnRight(){
• //code to turn right
• }
• }

```

There can be two cars or there can be a hundred. Each will be called an instance of that object. Individual instances can be modified without affecting other parts of the program.

Core OOP Concepts

Data Hiding/Abstraction

When data is grouped into classes/objects, programmers have the option of making data private to certain classes. Therefore the outer world will see only what is required to use the class without having access to any unnecessary sometimes private information.

Data Encapsulation

Wrapping up logically related data and functions that operate on this data into bundles called classes.

Packages

Containers for logically related classes and interfaces bundled together.

Inheritance

When one class can be based on another class, or reuse that implementation. It can also add its own features to that implementation. In our example above the class Car could have inherited certain features from another class (called its 'super class' or 'base class') called 'Vehicle'.

Polymorphism

Polymorphism is the ability for a method in Java to be able to do different things depending on the object it is acting upon. For example a method called `accelerate()` in our Car class could have different implementations depending on the individual Car object or depending on the parameters provided to it.